YILDIZ TECHNICAL UNIVERSITY
APPLIED SQL (MTM4692)
PROJECT MILESTONE

Textile Warehouse Stock Management System

Team No: 32

Adil Topçu 21052036

Mert Kılıç 21058031

1.INTRODUCTION

This project aims to develop a simple web-based warehouse stock management system for a textile company. The system will be used only by warehouse employees to add products, update stock levels, and record stock movements. Its main purpose is to provide a more organized and reliable way of tracking inventory, reducing errors and improving efficiency compared to manual methods.

2. PROBLEM STATEMENT

In many textile companies, stock tracking is often done manually, which can cause errors, missing records, and confusion. Managing products with different sizes, colors, and categories makes this process even more difficult. This project aims to solve these issues by providing a system that allows warehouse employees to manage stock easily and maintain accurate inventory records.

3.IMPORTANCE OF THE PROJECT

Warehouse stock management is essential for business efficiency. Accurate tracking prevents shortages, overstocking, and errors. This system offers a simple solution that improves accuracy, saves time, and supports better decision-making.

4.DATABASE DESING

The database is designed to manage warehouse stock operations efficiently. It focuses on product management, stock tracking, suppliers, and stock movements. The system consists of six main tables: employees, categories, suppliers, products, product_variants, and stock_movements, each serving a specific function.

4.1 Employees Table

This table stores the login and identification information of warehouse employees who use the system.

- employee_id – unique ID for each employee
- full_name – employee's full name
- username – login username
- password – login password

This table is necessary because only authorized warehouse staff can access the system and perform stock operations.

4.2 Categories Table

This table stores the categories of textile products, such as t-shirts, pants, jackets, or shirts.

- category_id – unique ID for each category
- category_name – name of the category

This table helps classify products and makes listing and filtering easier.

4.3 Suppliers Table

This table stores information about the suppliers that provide products to the warehouse.

- supplier_id – unique ID for each supplier
- supplier_name – supplier company name
- phone – contact number
- email – supplier email address
- address – supplier address

This table is important because products may be supplied by different suppliers.

4.4 Products Table

This table stores the main information about products in the warehouse.

- product_id – unique ID for each product
- product_name – name of the product
- category_id – category of the product
- supplier_id – supplier of the product
- brand – product brand
- price – unit price of the product

This table contains the general information of each product, but size and color details are stored separately in the product_variants table.

4.5 Product Variants Table

This table stores the variations of products based on size and color. This is especially important in textile products, because the same product can exist in multiple combinations such as different sizes and colors.

- variant_id – unique ID for each product variant
- product_id – related product
- size – product size
- color – product color
- stock_quantity – current stock amount of that specific variant

This table makes the system more realistic, because stock is tracked according to specific product variations instead of only general product information.

4.6 Stock Movements Table

This table records all stock entry and stock exit operations. It keeps track of which employee performed the action, on which product variant, in what quantity, and at what time.

- movement_id – unique ID for each stock movement
- variant_id – related product variant
- employee_id – employee who performed the action
- movement_type – type of movement (ENTRY or EXIT)
- quantity – amount of stock added or removed
- movement_date – date and time of the movement

This table is one of the most important parts of the system because it allows all stock changes to be recorded and monitored.

4.7 Relationships Between Tables

The tables are connected through primary key and foreign key relationships.

- Each product belongs to one category
- Each product is supplied by one supplier
- Each product can have multiple variants
- Each stock movement belongs to one product variant
- Each stock movement is performed by one employee

The relationships can be summarized as follows:

- categories → products = One-to-Many
- suppliers → products = One-to-Many
- products → product_variants = One-to-Many
- product_variants → stock_movements = One-to-Many
- employees → stock_movements = One-to-Many

This design enables the system to organize product, supplier, and stock data efficiently, including stock movement history. SQL features are used to build the database structure, manage table relationships, and perform data operations effectively.

5. THE FALLOWING SQL FEATURES WILL BE USED IN THE PROJECT:

CREATE TABLE – to create database tables such as employees, products, and stock movements

PRIMARY KEY – to uniquely identify each record in a table

FOREIGN KEY – to establish relationships between tables

NOT NULL – to ensure that required fields are not left empty

UNIQUE – to prevent duplicate values in specific columns (e.g., username)

DEFAULT – to assign default values (e.g., movement date)

INSERT – to add new records (e.g., adding a new product or stock entry)

UPDATE – to modify existing records (e.g., updating stock quantity)

DELETE – to remove records when necessary

SELECT – to retrieve data from the database

WHERE – to filter records based on conditions

JOIN – to combine data from multiple tables (e.g., products with categories)

ORDER BY – to sort query results

GROUP BY – to group data for analysis

COUNT(), SUM() – to perform aggregate calculations

These SQL features will allow the system to store, update, and retrieve data effectively while maintaining data integrity and consistency.

6. SAMPLE QUERIES

Query 1 – Add a Stock Entry (Product Entry)

This query is used when new products arrive at the warehouse and stock needs to be increased.

```
INSERT INTO stock_movements (variant_id, employee_id, movement_type, quantity, movement_date)

VALUES (1, 2, 'ENTRY', 50, CURRENT_TIMESTAMP);
```

Query 2 – Update Stock Quantity

This query updates the stock quantity of a product variant after a stock entry or exit operation.

```
UPDATE product_variants

SET stock_quantity = stock_quantity + 50

WHERE variant_id = 1;
```

Query 3 – Low Stock Products

```
SELECT p.product_name, pv.size, pv.color, pv.stock_quantity

FROM product_variants pv

JOIN products p ON pv.product_id = p.product_id

WHERE pv.stock_quantity < 10;
```